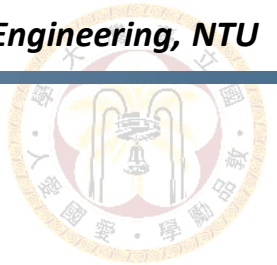
Computer-Aided VLSI System Design

Homework 3: Single-Channel Convolution with Barcode Decoding Engine

Graduate Institute of Electronics Engineering, National Taiwan University

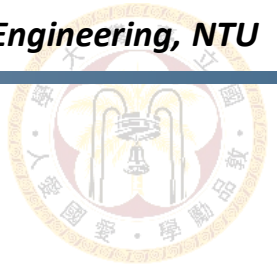


NTU GIEE



Goal

- In this homework, you will learn
  - How to synthesize your design
  - How to run gate-level simulation
  - How to integrate SRAM to your design
  - How to trade of between area and timing
  - How to generate test-pattern

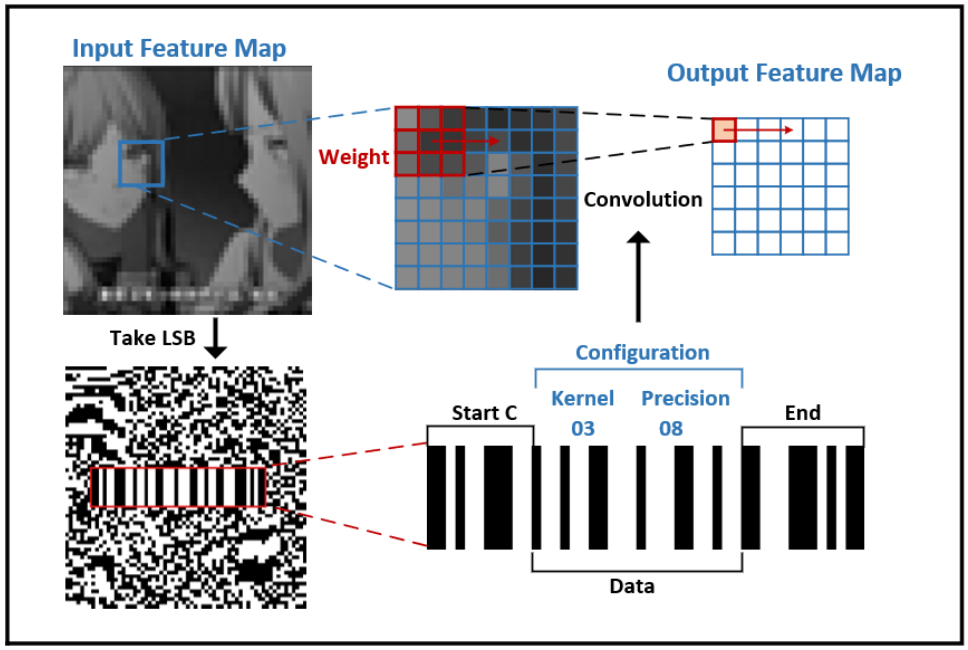


Introduction

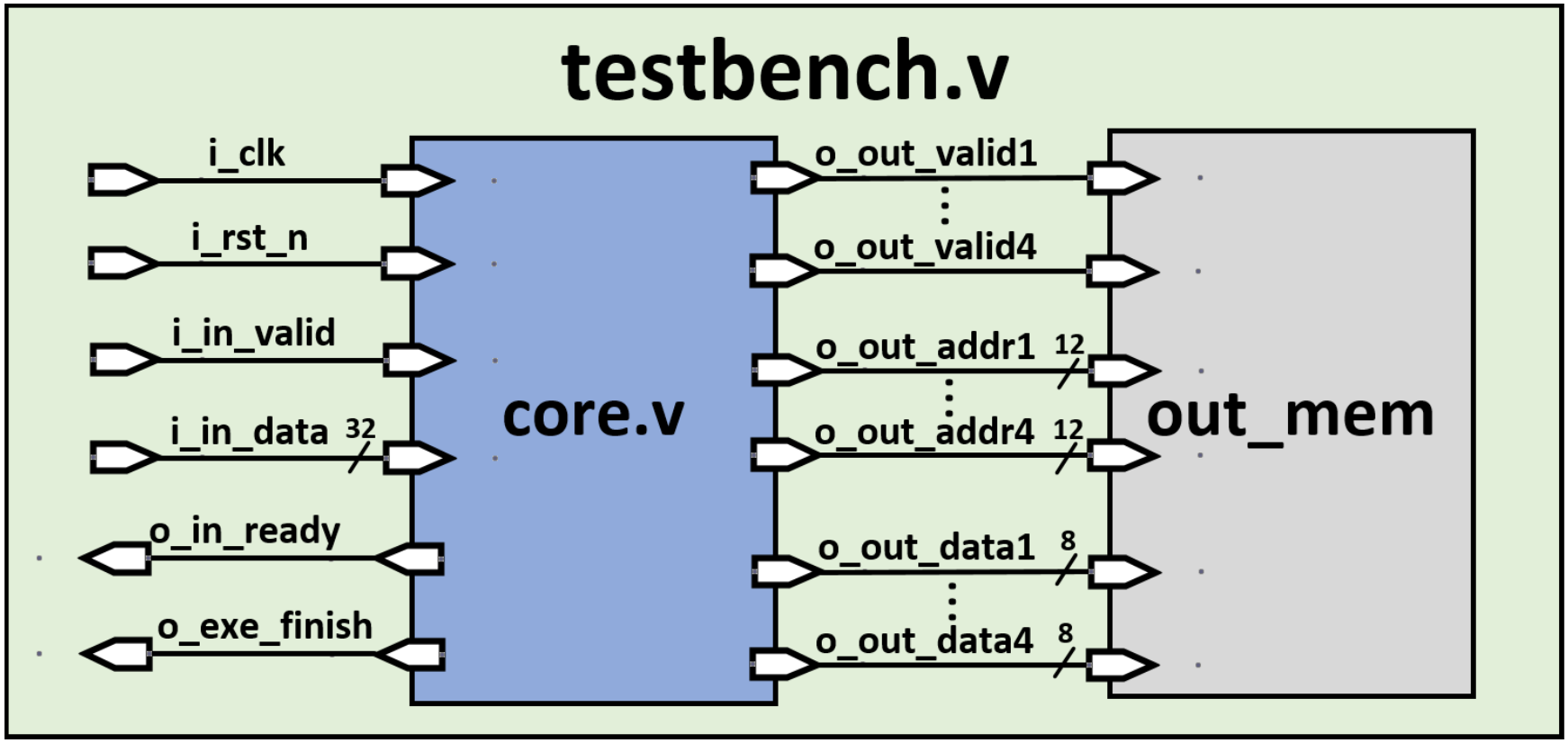
- Your client used to be a convenience store clerk. He asked you to design a convolution engine
  - Operation parameters are hidden within barcodes.
- In this homework, you are going to implement a single-channel convolution with Code 128-C barcode decoding engine

Workflow Overview

- A 64×64 grayscale image (8-bit per pixel) will be loaded as input.
- A **Code 128-C barcode** is embedded in its **least significant bits**
- The convolution operation should be performed under different configurations, as specified by the hyperparameters encoded in the barcode



Block Diagram



Input/Output

Signal Name	I/O	Width	Simple Description
i_clk	I	1	Clock signal in the system.
i_rst_n	I	1	Active **low asynchronous** reset.
i_in_valid	I	1	This signal is **high** if input data is valid
i_in_data	I	32	Input image/weight data
o_in_ready	O	1	Set to **high** if ready to get **next 4** input data
o_out_valid1-4	O	1	Set to **high** with valid output data
o_out_addr1-4	O	12	Address of output data
o_out_data1	O	8	Output data / Kernel size
o_out_data2	O	8	Output data / Weight precision
o_out_data3	O	8	Output data
o_out_data4	O	8	Output data
o_exe_finish	O	1	Set to **high** if the execution is finished

Specification (1)

- All inputs are synchronized with the **negative** edge clock
- All outputs should be synchronized at clock **rising** edge
- You should reset all your outputs when i_rst_n is **low**
  - Active low asynchronous reset is used and only once

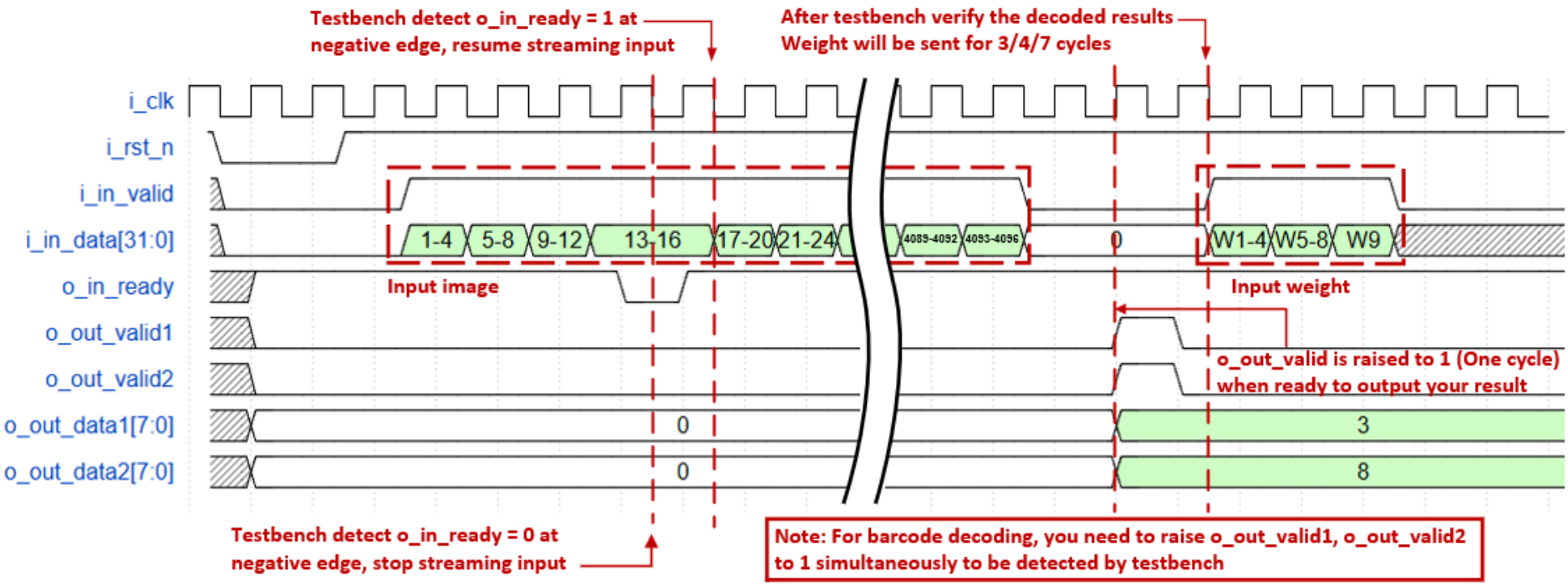
Specification (2)

- i_in_valid and o_out_valid can't be **high** at the same time
- In covolution stage, any two outputs **can't** be written to the **same address at the same time**
- o_out_valid should be **high** for valid output results
- Pull **o_exe_finish** to **high** after finish all the operation
- **At least one SRAM** is implemented in your design

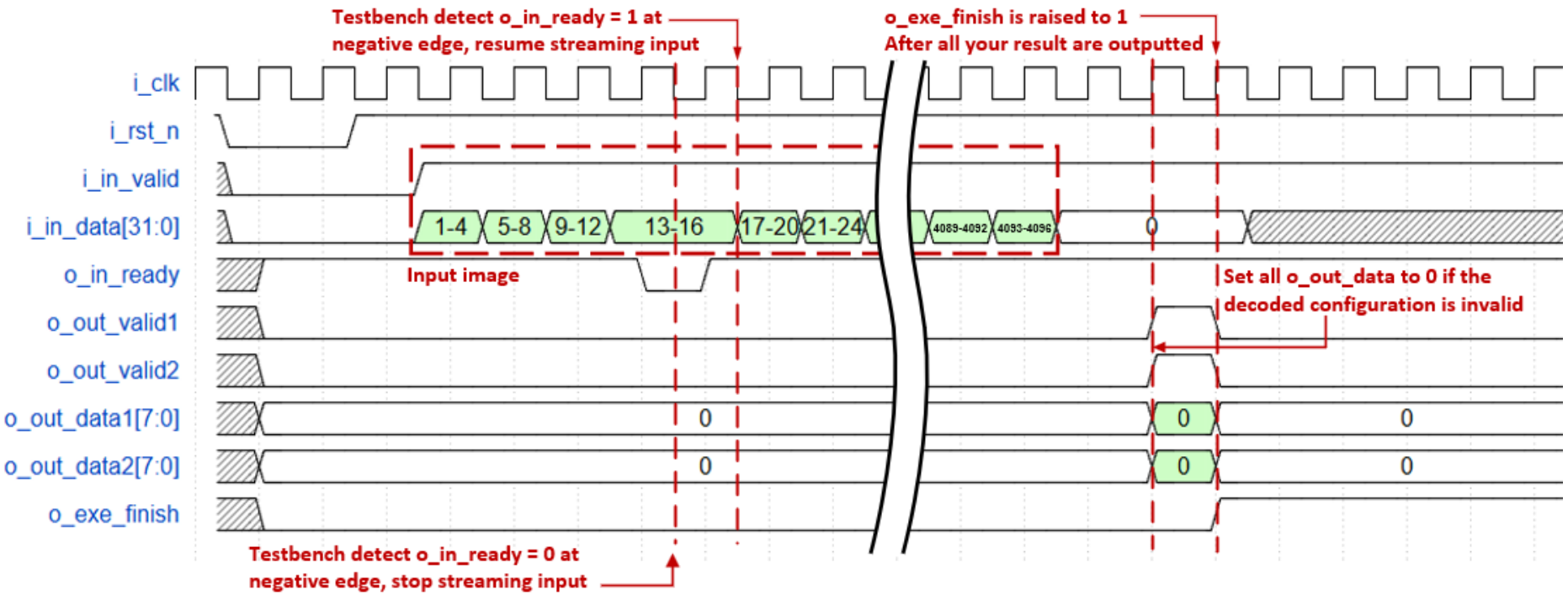
Specification (3)

- Only worst-case library is used for synthesis
- The synthesis result of data type should **NOT** include any **Latch**
- The slack for setup time in **synthesis timing report** should be **non-negative (MET)**
- **No timing violations** in the gate-level simulation **after reset**

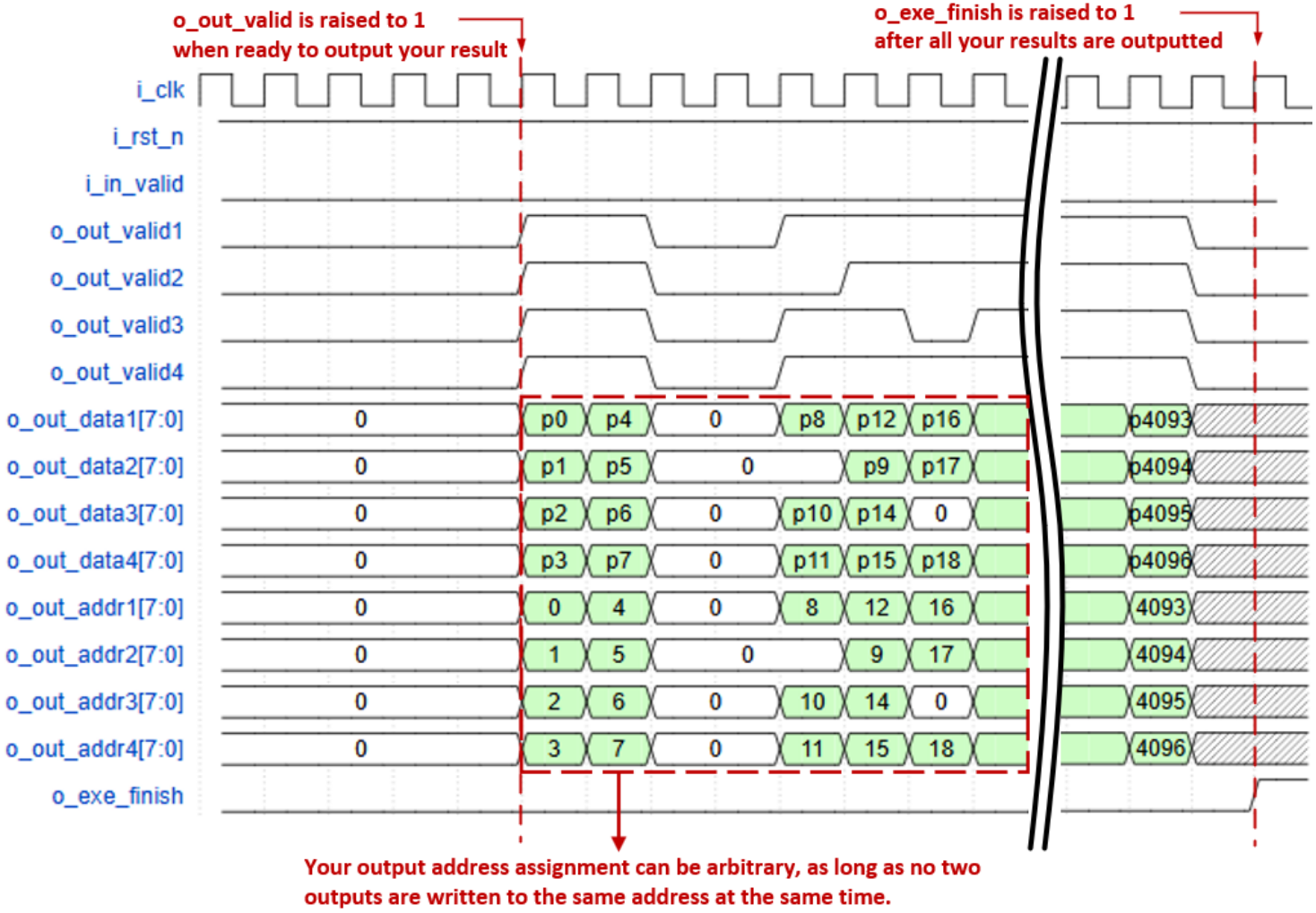
Waveform: Loading Image & Weight



Waveform: Load Image (Invalid Configuration)

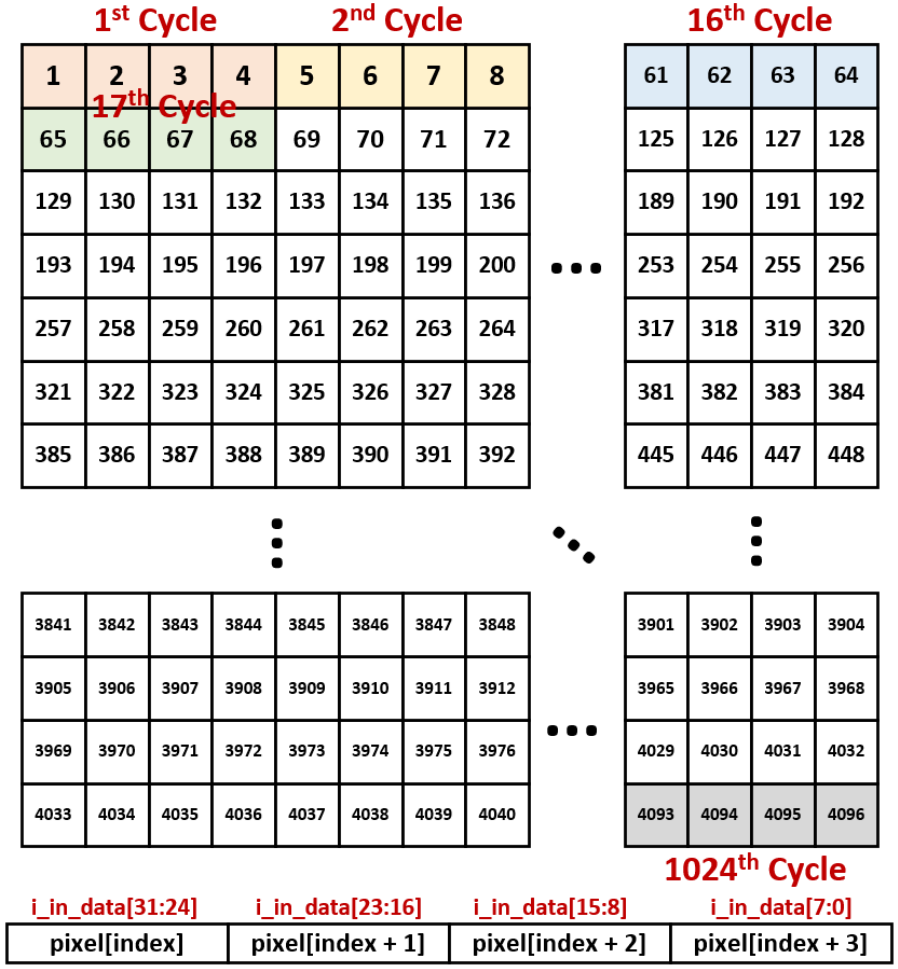


Waveform: Output of Convolution



Input Image

- The input image is given in **raster-scan** order

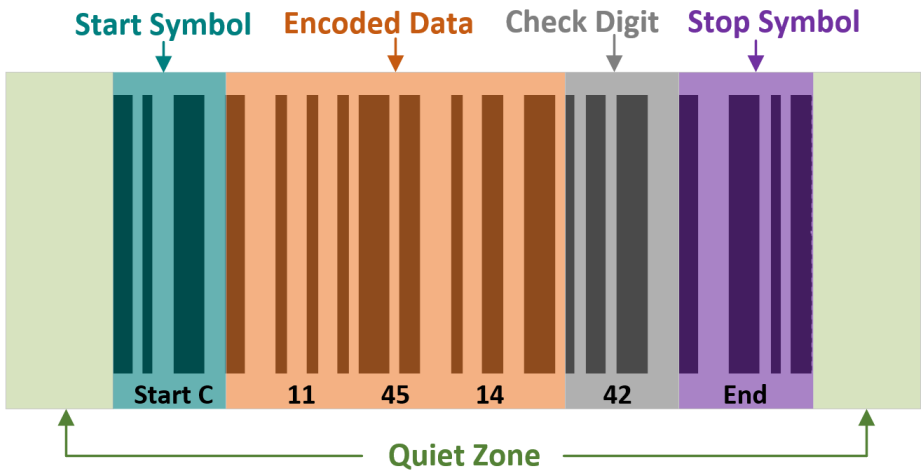


Input Image Loading

- An 64x64 image is loaded for 1024 cycles in **raster-scan** order
- The size of each pixel is 8 bits (**unsigned**)
- If **o_in_ready** is 0, stop input data until **o_in_ready** is 1
- The input feature map will be loaded only once at the beginning

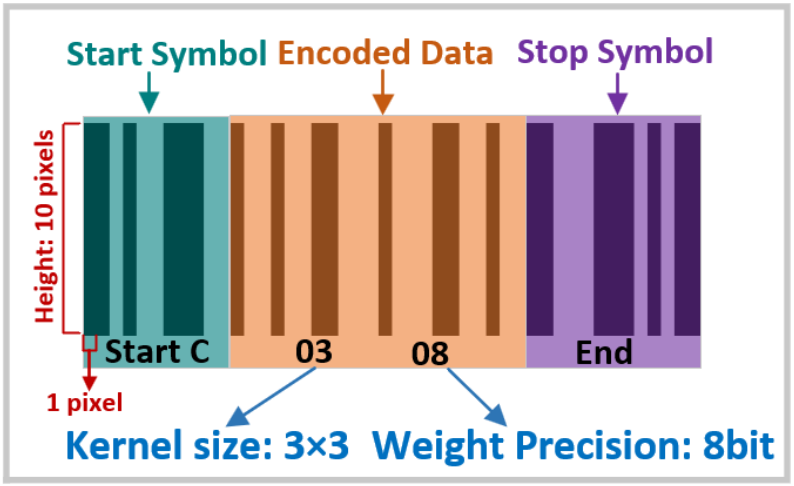
Code 128 Barcode

- **CODE 128** is a high-density linear barcode symbology capable of encoding all 128 ASCII characters, including control codes such as *[ESC]* and *[CR]*.
- A typical CODE 128 barcode consists of five sections:
  - Quiet zone (Not used in this homework)
  - Start Symbol
  - Encoded data
  - Check digit (Not used in this homework)
  - Stop symbol



Code 128-C Barcode

- A **Code 128-C barcode** with a height of 10 pixels is embedded within the **least significant bits (LSBs)** of input image.
- The barcode sequentially encodes two hyperparameters in following order:
  - Kernel size (*K*): limited to {03, 04, 05}
  - Weight Precision *(W)*: limited to {04, 08}
- The barcode always start with **Start C** and end with **Stop Symbol**



Details of Barcode Decoding

- **After loading the image**, your design must output the **2** hyperparameters **simultaneously**
- Use **o_out_data1**, **o_out_data2**, to output the decoded hyperparameters:
  - o_out_data1: *Kernel size (K)*
  - o_out_data2: *Weight Precision (W)*
- If the decoded hyperparameters do not match any of the valid combinations specified before (***K ∈{3, 4 ,5}, W ∈{4, 8}***)
  - Configuration is considered **invalid**, all outputs should be set to **0**.

Details of Barcode Decoding (cont.)

- The barcode can appear **anywhere in the image** but will always **stay fully inside the image boundaries**
- For convenience, a partial Code 128-C table is provided below for reference:

Value	128-C	Pattern	Widths
0	00	11011001100	212222
1	01	11001101100	222122
2	02	11001100110	222221
3	03	10010011000	121223
4	04	10010001100	121322
5	05	10001001100	131222
6	06	10011001000	122213
7	07	10011000100	122312
8	08	10001100100	132212
9	09	11001001000	221213
105	Start Code C	11010011100	211232
—	Stop Symbol	1100011101011	2331112

Introduction to Convolution

- Convolution is widely used in image processing and computer vision applications
- Applications:

Image Recognition

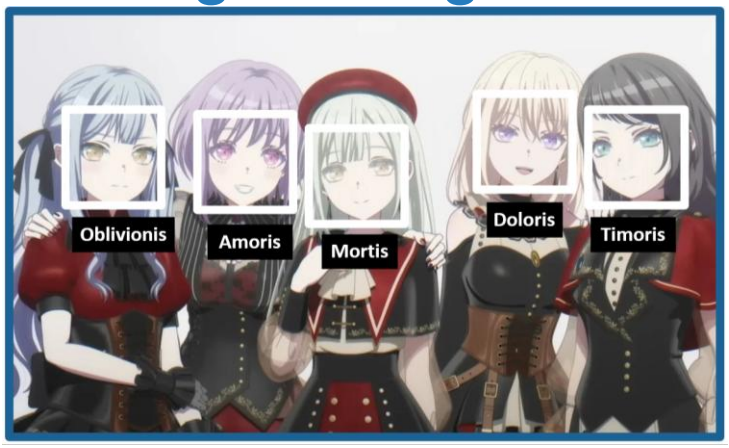


Image Segmentation



Source: https://ani.gamer.com.tw/

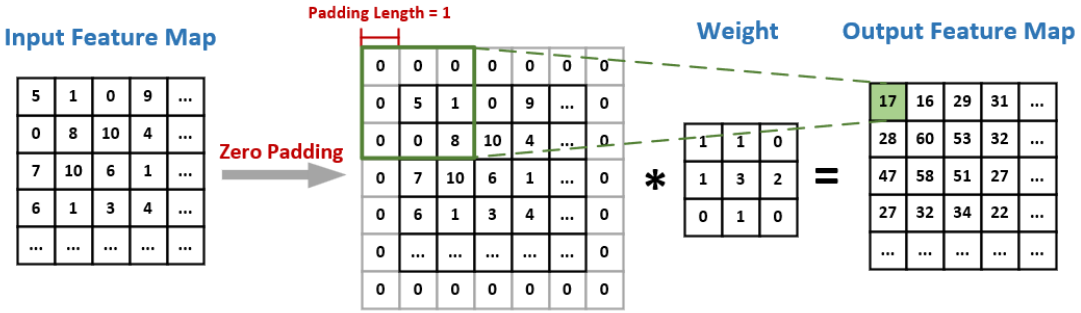
- For this homework, you have to implement a single channel convolution processor with 3×3 / 4x4 / 5x5 kernel size, supporting different weight precision

Convolution Operation

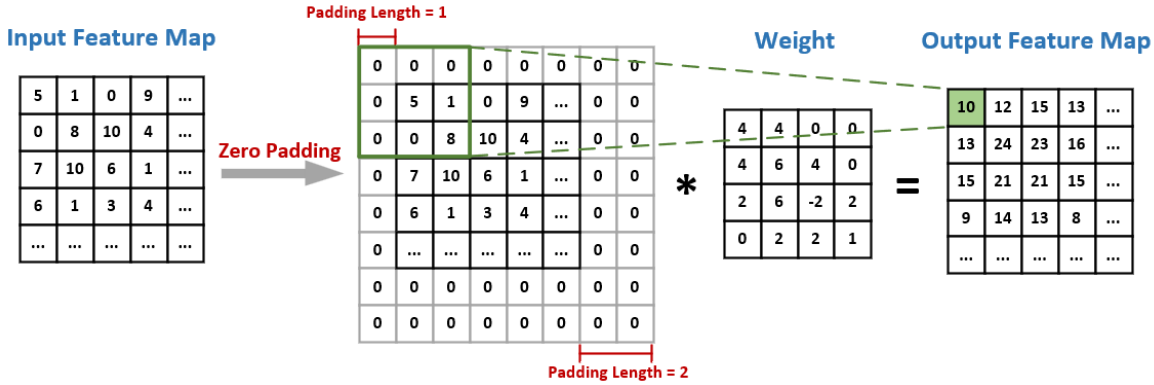
- For this homework, you have to perform convolution **over the entire image**
- **Zero padding** is applied to maintain a 64×64 output feature map
  - Top / Left padding = *floor((K − 1) / 2)*
  - Bottom / Right padding = *ceil((K − 1) / 2)*
- The accumulation results should be **rounded to the nearest integer** (“Round” mode in [1])
  - Do not truncate temporary results during computation
- After rounding, the final results should be **clamped to [0, 255]**
- The clamped final results of convolution should be written to the output memory
  - Output address is determined by **o_out_addr1 - 4**.

Example of Convolution

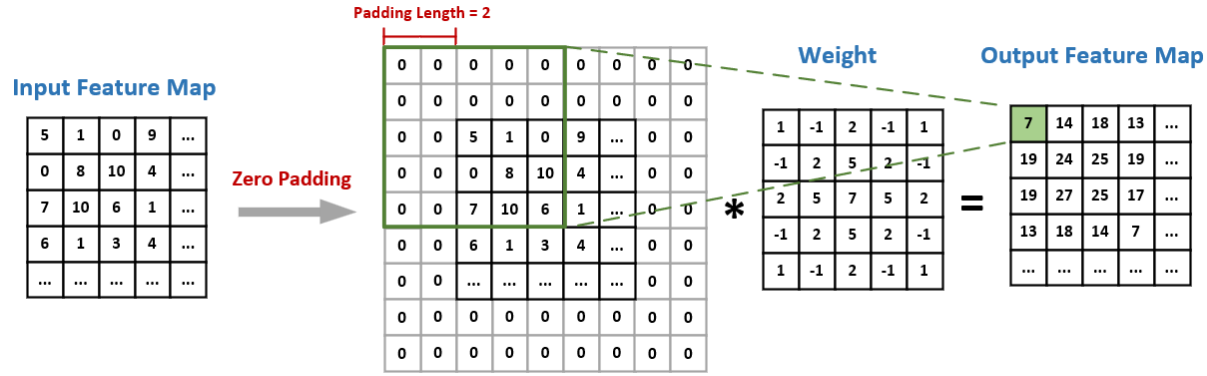
- Kernel size = 3



- Kernel size = 4

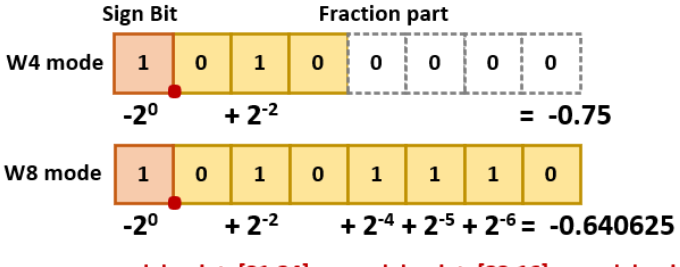


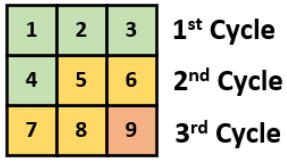
- Kernel size = 5



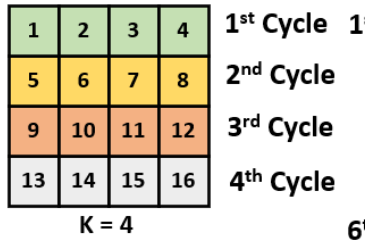
Weight Format

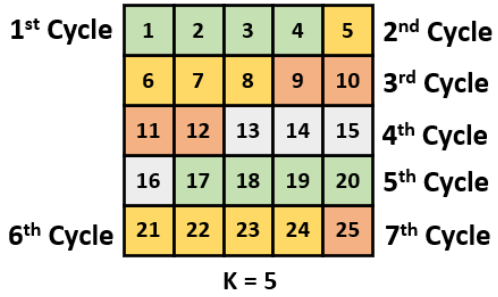
- The size of the kernel is 3×3 / 4×4 / 5×5
  - Determined by Kernel size (K)
- After the testbench verifies the config., K×K weights are loaded in 3 / 4 / 7 accepted cycles for K = 3 / 4 / 5, respectively
- Weights are stored in 4bits or 8bits signed fixed point number
  - Determined by weight precision (W)
  - Represented by 2's complement

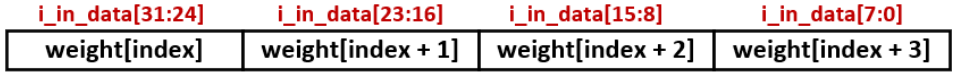


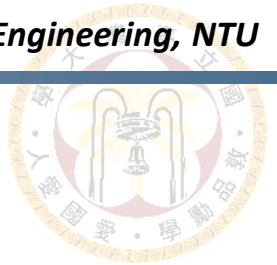


Weight Loading Order









About Convolution Outputs

- The output address is defined in **raster-scan** sequence for all operation
- The actual write order can be arbitrary as long as **no two outputs are written to the same address at the same time**
- After all your results are outputted, you should pull **o_exe_finish** to **high** so the testbench will then check all your results.

Testbench

```
`timescale 1ns/1ps

`ifndef CYCLE
    `define CYCLE 5.0
`endif
`define MAX_CYCLE 500000
`define RST_DELAY 2
`define OUTPUT_SIZE 4096
`define SDFFILE "../02_SYN/Netlist/core_syn.sdf"

// tb0 is the invalid-barcode case. tb1--tb6 cover every K/precision pair.
`ifdef tb0
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb0.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb1.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb1.dat"
    `define K_SIZE 0
    `define W_PRECISION 0
    `define WEIGHT_COUNT 0
    `define VALID_OP 0
`elsif tb2
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb2.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb2.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb2.dat"
    `define K_SIZE 3
    `define W_PRECISION 8
    `define WEIGHT_COUNT 9
    `define VALID_OP 1
`elsif tb3
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb3.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb3.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb3.dat"
    `define K_SIZE 4
    `define W_PRECISION 4
    `define WEIGHT_COUNT 16
    `define VALID_OP 1
```

```
`elsif tb4
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb4.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb4.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb4.dat"
    `define K_SIZE 4
    `define W_PRECISION 8
    `define WEIGHT_COUNT 16
    `define VALID_OP 1
`elsif tb5
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb5.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb5.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb5.dat"
    `define K_SIZE 5
    `define W_PRECISION 4
    `define WEIGHT_COUNT 25
    `define VALID_OP 1
`elsif tb6
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb6.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb6.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb6.dat"
    `define K_SIZE 5
    `define W_PRECISION 8
    `define WEIGHT_COUNT 25
    `define VALID_OP 1
`else
    // Default to tb1: K=3, W4.
    `define INFILE  "../00_TESTBED/PATTERNS/image_tb1.dat"
    `define WFILE   "../00_TESTBED/PATTERNS/weight_tb1.dat"
    `define GOLDEN  "../00_TESTBED/PATTERNS/golden_tb1.dat"
    `define K_SIZE 3
    `define W_PRECISION 4
    `define WEIGHT_COUNT 9
    `define VALID_OP 1
`endif
```

```
//For gate-level simulation only
`ifdef SDF
    initial $sdf_annotate(`SDFFILE, u_core);
    initial #1 $display("SDF File %s were used for this simulation.", `SDFFILE)
`endif
```

Pattern

- **7 public test cases** are provided for grading
  - Provided information includes
    - input image, image LSB map, weight and golden output
  - Hidden patterns use different images, weights, and barcode positions

image_tb1.dat

```
AF
B9
AC
83
7C
84
85
99
91
8A
90
9B
9C
9E
9F
90
```

weight_tb1.dat

```
20
40
30
20
20
10
10
20
D0
```

golden_tb1.dat

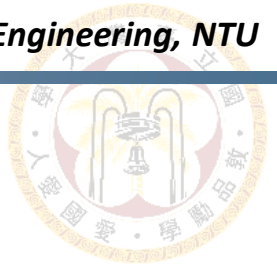
```
2B
76
78
5E
48
55
56
55
60
5B
56
5B
5E
63
68
61
```

image_tb1.png



lsb_tb1.png





01_RTL

- **core.v**

```
module core (                       //Don't modify interface
    input           i_clk,
    input           i_rst_n,
    input           i_in_valid,
    input   [31: 0] i_in_data,

    output          o_in_ready,

    output  [ 7: 0] o_out_data1,
    output  [ 7: 0] o_out_data2,
    output  [ 7: 0] o_out_data3,
    output  [ 7: 0] o_out_data4,

    output  [11: 0] o_out_addr1,
    output  [11: 0] o_out_addr2,
    output  [11: 0] o_out_addr3,
    output  [11: 0] o_out_addr4,

    output          o_out_valid1,
    output          o_out_valid2,
    output          o_out_valid3,
    output          o_out_valid4,

    output          o_exe_finish
);
```

01_RTL

- **rtl_01.f**

```
// ------------------------------------------------------------------------------
// Simulation: HW3
// ------------------------------------------------------------------------------

// testbench
// ------------------------------------------------------------------------------
../00_TESTBED/testbench.v

// memory file
// ------------------------------------------------------------------------------
//../sram_256x8/sram_256x8.v
//../sram_512x8/sram_512x8.v
//../sram_4096x8/sram_4096x8.v


// design files
// ------------------------------------------------------------------------------
./core.v
```

01_RTL

- Run the RTL simulation under 01_RTL folder

```
vcs -f rtl_01.f -full64 -R -debug_access+all +v2k
+notimingcheck -sverilog +define+tb0
```

tb0, tb1, tb2, tb3, tb4, tb5, tb6

or

```
./01_run tb0 5.0
```

clock period

01_RTL

- SpyGlass linting

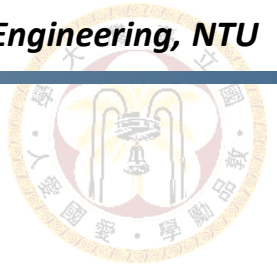
```
./02_lint
```

- Command for cleaning temporary files

```
./99_clean_up
```

- Note that before executing the shell script, change the file permissions by

```
chmod +x ./01_run ./02_lint ./99_clean_up
```



02_SYN

- **core_dc.sdc**

```
# operating conditions and boundary conditions #
set cycle  5.0;  # modify your clock cycle here #
```

- **flist.sv**

```
1   // list all paths to your design files
2   `include "../01_RTL/core.v"
```

- Run the command to do synthesis

```
dc_shell-t -f syn.tcl | tee syn.log
```

03_GATE

- Run gate-level simulation under 03_GATE folder

```
vcs -f rtl_03.f -full64 -R -debug_access+all +v2k
+maxdelays -negdelay +neg_tchk +define+SDF+tb0
```

tb0, tb1, tb2, tb3, tb4, tb5, tb6

or

```
./03_run tb0 5.0
```

clock period

sram_256x8

Pin Description

Pin	Description
A[7:0]	Addresses (A[0] = LSB)
D[7:0]	Data Inputs (D[0] = LSB)
CLK	Clock Input
CEN	Chip Enable
WEN	Write Enable
Q[7:0]	Data Outputs (Q[0] = LSB)

Area

Area Type	Width (mm)	Height (mm)	Area (mm^2)
Core	0.143	0.125	0.018
Footprint	0.153	0.136	0.021

The footprint area includes the core area and user-defined power ring and pin spacing areas.

SRAM Logic Table

CEN	WEN	Data Out	Mode	Function
H	X	Last Data	Standby	Address inputs are disabled; data stored in the memory is retained, but the memory cannot be accessed for new reads or writes. Data outputs remain stable.
L	L	Data In	Write	Data on the data input bus D[n-1:0] is written to the memory location specified on the address bus A[m-1:0], and driven through to the data output bus Q[n-1:0].
L	H	SRAM Data	Read	Data on the data output bus Q[n-1:0] is read from the memory location specified on the address bus A[m-1:0].

Submission

- Create a folder named **studentID_hw3** and follow the hierarchy below

```
r13943017_hw3/
    |— 01_RTL/
    |       |— core.v (and other Verilog files)
    |       |__ rtl_01.f
    |— 02_SYN/
    |       |— syn.tcl
    |       |— flist.sv
    |       |— core_dc.sdc
    |       |— Netlist/
    |       |      |— core_syn.v
    |       |      |— core_syn.sdf
    |       |      |__ core_syn.ddc
    |       |__ Report/
    |              |— core_syn.area
    |              |__ core_syn.timing
    |— 03_GATE/
    |       |__rtl_03.f
    |
    |__ report.txt
```

- Compress the folder **studentID_hw3** in a tar file named **cvsd_hw3_v*k*.tar** (*k* is the number of version, *k* =1,2,…)
  - Use lower case for the letter in your student ID

Grading Policy

- Correctness of simulation: **70%** (follow our spec)

Pattern	Description	RTL simulation	Gate-level simulation
tb0	Barcode decoding (invalid configuration)	6%	6%
tb1	Barcode decoding + convolution (K=3,W=4)	4%	4%
tb2	Barcode decoding + convolution (K=3,W=8)	4%	4%
tb3	Barcode decoding + convolution (K=4, W=4)	4%	4%
tb4	Barcode decoding + convolution (K=4,W=8)	4%	4%
tb5	Barcode decoding + convolution (K=5,W=4)	4%	4%
tb6	Barcode decoding + convolution (K=5,W=8)	4%	4%
tbh	Hidden patterns	x	10%

*K means kernel size, W means weight precision

- Performance: **30%**
  - Performance = **Area * Time (μm^2 * ns)**
    - **Time = total simulation time of tb1 + tb2 + tb3 + tb5 (image loading is excluded)**
  - Performance score only counts if your design passes all the test patterns

Grading Policy

- **No late submission is allowed**
  - Any submissions after the deadline will receive 0 point
- Lose **5 points** for any wrong naming rule or format for submission
  - Ensure that the submitted files can be decompressed and executed without issues
- **No plagiarism**
  - Plagiarism in any form, including copying from online sources and sharing AI prompt with others, is strictly prohibited

Grading Policy

- **Violations of any spec (p.7 – p.9) incur point penalties**
  - Negative slack (**Violate** in synthesis timing report)
    - 0 point for gate-level simulations and performance
  - Design without SRAM
    - 0 point for gate-level simulations and performance
  - Violate other rules but pass all simulations
    - Performance score * 0.7
- You should NOT use the **improper method** to finish the homework, we have the right to deduct your score
  - Try to copy and paste the golden pattern to your code
  - Make the synthesis circuit and RTL code mismatch deliberately
- If we find some weird thing in your code, you must explain your purpose, otherwise, you may lose your score

Area

- core_syn.area

```
Number of ports:                        1058
Number of nets:                        30734
Number of cells:                       27064
Number of combinational cells:         23297
Number of sequential cells:             3721
Number of macros/black boxes:              8
Number of buf/inv:                      3032
Number of references:                      7

Combinational area:            241592.639554
Buf/Inv area:                   17281.229314
Noncombinational area:         128951.475231
Macro/Black Box area:          247583.093750
Net Interconnect area:        3140029.522614

Total cell area:               618127.208535
Total area:                   3758156.731149
```

Number of macros/black boxes should not be 0

618127.208535 μm^2

Time

- **Simulation Result**

```
------------------------------------------------
         STAGE 1:  BARCODE DECODING
------------------------------------------------
All Configurations Correct! Permission Granted to Enter STAGE 2
------------------------------------------------
         STAGE 2:  CONVOLUTION
------------------------------------------------

    ************************************
    *    OVERALL COMPARISON RESULTS    *
    ************************************

------------------------------------------------
Case: K=3, W4, valid=1
Result writes: 4096
Execution cycles: 1105
```

1105cycles

```
        #    ###############    _   _
       #     #             #    *   *
   #   #     #   CORRECT   #      |
    # #      #             #    \___/
     #       ###############

------------------------------------------------
       CONGRATULATION! ALL DATA PASS!
------------------------------------------------
Total SPEC Error: 0
$finish called from file "../00_TESTBED/testbench.v", line 447.
$finish at simulation time              10670000
          V C S   S i m u l a t i o n   R e p o r t
Time: 10670000 ps
```

Number of SPEC Error should be 0

Report

- Your design will be simulated using the clock period in report.txt

- **report.txt**

```
student ID: r13943017

Clock Period: 5.0 (ns)

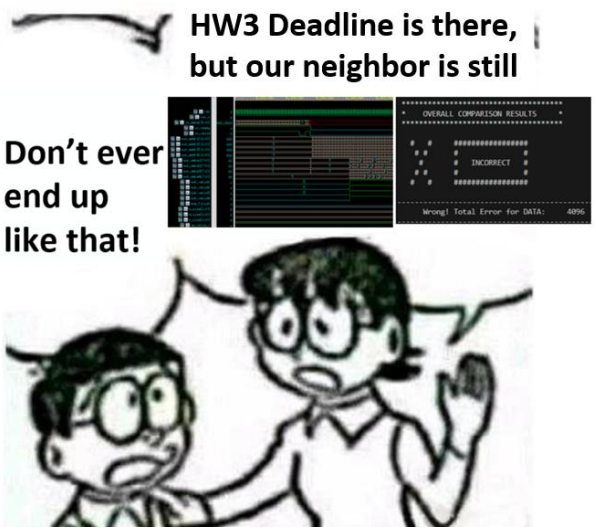
Execution cycle for tb1: 1105
Execution cycle for tb2: ...
Execution cycle for tb3: ...
Execution cycle for tb4: ...
Execution cycle for tb5: ...
Execution cycle for tb6: ...

Area:   618127.208535     (um^2)
```

The clock period that can pass all gate-level simulations without any timing violations

Notice

- **This homework is challenging**
  - You also have to prepare for **Midterm**
  - Make sure to start your RTL design earlier
- **Plan your hardware architecture before writing RTL**
  - "***Starting over is harder than starting up***"
    - *from "Getting over It with Bennett Foddy" 2017*



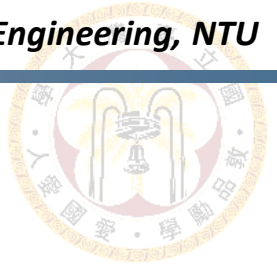
Some Guidelines for Better Performance

- Performance can be decomposed to Area * Cycle time * # of cycles
  - Hardware (multipliers) reuse
    - **Area↓**
  - Increasing **hardware utilization rate**, that is, make the idle cycles of multipliers as less as possible
    - **# of cycles↓ , Area ↓**
  - Do more pipelining, avoid long path
    - SRAM, Multipliers, In/output, etc.
    - **Cycle time ↓**
  - A more efficient memory mapping, data flow
    - **# of cycles↓, Area of control logic↓**

Discussion

- **NTU Cool Discussion Forum**
  - For any questions not related to assignment answers or privacy concerns, please use the NTU Cool discussion forum
  - **TAs will prioritize answering questions on the NTU Cool discussion forum**

- **Email:** r13943017@ntu.edu.tw
  - Title should start with **[CVSD 2026 Fall HW3]**
  - Email with wrong title will be moved to trash automatically



References

- [1] Rounding to the nearest
  - Rounding - MATLAB & Simulink (mathworks.com)
- [2] A guide to convolution arithmetic for deep learning
  - https://arxiv.org/abs/1603.07285
- [3] Code 128 wiki
  - https://en.wikipedia.org/wiki/Code_128